



LeetCode 700 — Search in a Binary Search Tree

1. Problem Title & Link

Title: LeetCode 700: Search in a Binary Search Tree

Link: <https://leetcode.com/problems/search-in-a-binary-search-tree/>

2. Problem Statement (Short Summary)

You are given the **root of a Binary Search Tree (BST)** and an integer **val**.

Your task:

👉 Find the **node** in the BST whose value equals **val** and return the **subtree rooted at that node**.

If such a node does **not** exist, return **null**.

📌 **BST Property:**

- Left subtree values < node.val
- Right subtree values > node.val

3. Examples (Input → Output)

Example 1

Input:

root = [4,2,7,1,3]

val = 2

Output:

[2,1,3]

Example 2

Input:

root = [4,2,7,1,3]

val = 5

Output:

null

4. Constraints

- Number of nodes: $1 \leq n \leq 5000$
- Node values: $1 \leq \text{val} \leq 10^7$
- Tree is a **valid BST**
- No duplicate values

5. Core Concept (Pattern / Topic)



Binary Search Tree Traversal (DFS / Iterative)

We exploit the **BST ordering property** to skip half the tree at every step.

6. Thought Process (Step-by-Step Explanation)

✗ Wrong Thinking

- Traversing the entire tree ✗
- Using DFS/BFS like a normal binary tree ✗

✓ Correct Thinking — Binary Search Logic

At any node:

- If $val == node.val \rightarrow$ **found**
- If $val < node.val \rightarrow$ search **left subtree**
- If $val > node.val \rightarrow$ search **right subtree**

This is exactly **binary search**, but on a tree.

7. Visual / Intuition Diagram (ASCII)

BST:

```
  4
 / \
2   7
 / \
1   3
```

Search $val = 2$:

4 $\rightarrow$ go left

2 $\rightarrow$ found ✓

8. Pseudocode (Language Independent)

```
function searchBST(node, val):
    if node == null:
        return null
    if node.val == val:
        return node
    if val < node.val:
        return searchBST(node.left, val)
    else:
        return searchBST(node.right, val)
```



9. Code Implementation

✓ Python (Recursive DFS)

```
class Solution:
    def searchBST(self, root, val):
        if not root:
            return None
        if root.val == val:
            return root
        if val < root.val:
            return self.searchBST(root.left, val)
        else:
            return self.searchBST(root.right, val)
```

✓ Java (Recursive DFS)

```
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        if (root == null) return null;
        if (root.val == val) return root;

        if (val < root.val)
            return searchBST(root.left, val);
        else
            return searchBST(root.right, val);
    }
}
```

✓ Java (Iterative — Interview Friendly)

```
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        while (root != null) {
            if (root.val == val) return root;
            if (val < root.val) root = root.left;
            else root = root.right;
        }
        return null;
    }
}
```



10. Time & Space Complexity

Time

- **$O(h)$** — height of BST
 - Balanced BST: $O(\log n)$
 - Skewed BST: $O(n)$

Space

- Recursive: **$O(h)$**
- Iterative: **$O(1)$**

11. Common Mistakes / Edge Cases

✗ Mistakes

- Ignoring BST property
- Searching both subtrees unnecessarily
- Forgetting to return subtree, not just value

✓ Edge Cases

- Empty tree
- Single node
- Value not present

12. Detailed Dry Run (Step-by-Step Table)

Input:

root = [4,2,7,1,3], val = 2

Node	Comparison	Action
4	$2 < 4$	go left
2	$2 == 2$	return node

Output subtree:

[2,1,3]

13. Common Use Cases (Real-Life / Interview)

- Database index search
- Range queries
- Ordered data lookup
- Auto-complete engines

14. Common Traps (Important!)

- Using BFS unnecessarily



- Not exploiting BST ordering
- Returning wrong subtree

15. Builds To (Related LeetCode Problems)

- LC 98 — Validate Binary Search Tree
- LC 701 — Insert into a BST
- LC 450 — Delete Node in a BST
- LC 230 — Kth Smallest Element in a BST

16. Alternate Approaches + Comparison

✓ 1 Recursive BST Search

- Clean
- Easy to explain

✓ 2 Iterative BST Search (BEST for space)

- No recursion stack
- Preferred in interviews

17. Why This Solution Works (Short Intuition)

BST ordering allows us to **discard half the tree** at each step.
We never search where the value cannot exist.

18. Variations / Follow-Up Questions

- Range search in BST
- Count nodes $\leq k$
- Find predecessor / successor
- Convert BST to sorted list
-